

LegacyLens Pre-Search Document

Completed before writing any code. All 16 checklist items across 3 phases.

Phase 1: Define Your Constraints

1. Scale & Load Profile

- **Target codebase:** LAPACK (Linear Algebra PACKage) — 600K+ LOC across ~1,700 Fortran files, ~3,500 subroutines. MVP indexes the full library; no partial-scope compromise needed given the small embedding cost.
- **Expected query volume:** Single-user CLI during development. Production projection: 5 queries/user/day.
- **Batch ingestion or incremental updates:** Batch only. LAPACK is a stable, versioned library — no live updates. Full re-index on demand if chunking strategy changes.
- **Latency requirements:** <3 seconds end-to-end per query (Retrieval < 500ms + LLM < 2s).

2. Budget & Cost Ceiling

- **Vector database hosting:** \$0/month — Pinecone free tier (5 indexes, 2GB storage, 100K read units/month).
- **Embedding API costs:** OpenAI text-embedding-3-small at \$0.02/1M tokens. Full LAPACK ingestion: ~2.4M tokens = ~\$0.05 one-time.
- **LLM API costs:** GPT-4o-mini — \$0.15/1M input, \$0.60/1M output. Per query (~3K input + ~500 output): ~\$0.00075.
- **Where will you trade money for time:** Everywhere. Managed services (Pinecone, OpenAI) over self-hosted alternatives. The sprint is 1 week — time is the binding constraint.

Item	Tokens	Cost
Embedding LAPACK (full ingestion)	~2.4M	\$0.05
Re-indexing experiments (3x)	~7.2M	\$0.15
Query embedding during dev (~200 queries)	~10K	<\$0.01
LLM answer generation during dev (~200)	~700K	\$0.15
Total development spend		~\$0.35

3. Time to Ship

- **MVP timeline:** 24 hours. Scope: ingestion pipeline + CLI retrieval with citations. Web UI is a Day 2+ stretch goal — not MVP.
- **Must-have:** Ingest LAPACK, chunk by subroutine, embed, store in Pinecone, semantic search via CLI, answer generation with file/line citations, syntax highlighting in terminal output.
- **Nice-to-have:** Web UI (Next.js/Vercel), re-ranking, dependency mapping, code explanation features, query expansion, caching.
- **Framework learning curve acceptable:** No. Custom Python pipeline — no time to learn LangChain abstractions, and generic text splitters (e.g., RecursiveCharacterTextSplitter) would destroy subroutine boundaries in Fortran.

4. Data Sensitivity

- **Is the codebase open source or proprietary:** Open source. LAPACK is a public Netlib project (BSD license).
- **Can you send code to external APIs:** Yes — no restrictions.
- **Data residency requirements:** None.

5. Team & Skill Constraints

- **Familiarity with vector databases:** Some — Pinecone is new but well-documented with a simple API.
- **Experience with RAG frameworks:** Limited. This is the learning objective. Custom pipeline to understand internals.
- **Comfort with target legacy language:** Low Fortran experience, but LAPACK has extremely well-structured subroutines with consistent naming conventions (DGESV, SGEMM, etc.) and clear SUBROUTINE/END SUBROUTINE boundaries, making syntax-aware parsing straightforward via regex.

Phase 2: Architecture Discovery

6. Vector Database Selection

Choice: Pinecone (managed cloud)

- **Managed vs self-hosted:** Managed. Zero ops burden in a 1-week sprint.
- **Filtering and metadata requirements:** Filter by file path, subroutine name, data type prefix (S/D/C/Z). Pinecone supports metadata filtering natively.
- **Hybrid search needed:** Not for MVP. Pure vector similarity is sufficient for subroutine-level chunks.
- **Scaling characteristics:** Free tier handles the full corpus easily (~3,500 vectors at 1536 dims < 50MB).

Alternative	Why not
ChromaDB	No managed hosting — would need a server
Weaviate	More complex setup, GraphQL API adds friction
Qdrant	Strong option but self-hosted adds ops burden
pgvector	Would need a Postgres instance to manage

7. Embedding Strategy

Choice: OpenAI text-embedding-3-small (1536 dimensions)

- **Code-specific vs general-purpose:** General-purpose. Fortran subroutines are heavily commented in English, so text-embedding-3-small handles them well. Voyage Code 2 is code-optimized but costs more and adds another API dependency for marginal gain on well-commented Fortran.
- **Dimension size tradeoffs:** 1536 dims — good balance. 3072 (text-embedding-3-large) doubles storage with marginal quality gain. 1024 (Cohere) loses fidelity.
- **Local vs API-based:** API-based. Local sentence-transformers would add dependency complexity and is slower without GPU. API cost for full LAPACK is ~\$0.05.
- **Batch processing:** Embed in batches of 100 chunks per API call. OpenAI supports batch embedding natively. Total ingestion: ~35 API calls.

8. Chunking Approach

Choice: Subroutine/function-level (Fortran natural boundaries)

- **Strategy:** Syntax-aware splitting via regex on SUBROUTINE/FUNCTION/PROGRAM keywords and their corresponding END statements.
- **Optimal chunk size:** Most LAPACK subroutines are 50–300 lines (~200–1200 tokens), well within the 8191 token limit for text-embedding-3-small.
- **Oversized chunk fallback:** Subroutines exceeding ~~1500 tokens get fixed-size splitting with 10% overlap~~ (50 tokens). This handles the handful of large routines (e.g., some driver routines are 500+ lines) without losing context at split boundaries.
- **Overlap strategy:** No overlap for subroutine-level chunks — each is a self-contained unit. Overlap only applies to the fixed-size fallback above.
- **Metadata to preserve:** File path, subroutine name, line numbers (start/end), parameter list, data type prefix (S/D/C/Z for single/double/complex/double-complex), dependencies (CALL statements extracted via regex).

9. Retrieval Pipeline

- **Top-k value:** k=5 for initial retrieval. Returns the 5 most similar chunks.
- **Metadata filtering at query time:** Filter Pinecone results by data_type (S/D/C/Z) and category (BLAS vs LAPACK) to improve precision and reduce noisy returns.
- **Re-ranking:** None for MVP. Pinecone cosine similarity scoring is the ranking. Cross-encoder re-ranking is a stretch goal.
- **Context window management:** 5 chunks x ~800 tokens avg = ~4000 tokens of context, well within GPT-4o-mini's 128K window. Each chunk is injected with its metadata header (file path, line numbers, subroutine name).
- **Multi-query or query expansion:** Not for MVP. Single query embedding, single Pinecone search.

10. Answer Generation

Choice: GPT-4o-mini

- **Why this model:** Excellent cost/quality ratio. Fast responses (~1–2s). Sufficient reasoning for code explanation. GPT-4o would be 10x more expensive with marginal benefit for this task.
- **Prompt template:** System prompt establishes the LLM as a Fortran/LAPACK expert. Includes explicit instruction: "Answer using ONLY the provided context snippets. For every factual claim or code reference, you MUST cite the file path and line number. If the context is insufficient, say so — do not fabricate."
- **Citation format:** Inline references like [file.f:L42-L87]. Retrieved chunks displayed below the answer with syntax highlighting.
- **Streaming:** Yes. Streaming to CLI for responsive feel. Next.js streaming via OpenAI SDK when web UI is added post-MVP.

11. Framework Selection

Choice: Custom Python pipeline (no framework)

Rationale:

- LAPACK ingestion is a one-time batch job — no need for chain abstractions
- The pipeline is straightforward: embed query -> Pinecone search -> assemble context -> LLM call
- Custom code is easier to debug in a 1-week sprint
- LangChain's generic text splitters would destroy Fortran subroutine boundaries
- Understanding RAG internals is the learning objective

Integration: Python ingestion script outputs to Pinecone. CLI reads from Pinecone + calls OpenAI. No framework coupling between ingestion and serving.

Phase 3: Post-Stack Refinement

12. Failure Mode Analysis

- **No relevant results:** If the top similarity score < 0.7 threshold, return "No relevant code found in indexed subroutines" with a suggestion to rephrase. The 0.7 threshold is appropriate for code retrieval — lower thresholds (e.g., 0.3) return noisy, irrelevant chunks that degrade answer quality.
- **Ambiguous queries:** Return top-k results with scores and let the user evaluate. The LLM answer includes a confidence qualifier. Short/vague queries prompt the user to be more specific.
- **Hallucination prevention:** The system prompt explicitly forbids answering without context. If retrieved chunks seem irrelevant to the query, the LLM must say so rather than fabricate.
- **Rate limiting and errors:** OpenAI API errors (429, 500) get retried once with exponential backoff via tenacity, then return a user-friendly error. Pinecone timeouts produce a "service temporarily unavailable" message.

13. Evaluation Strategy

- **Retrieval precision:** Curate 10 test queries with known-correct subroutines as ground truth. Measure precision@5 (fraction of top-5 results that are relevant). Target: >70%.
- **Ground truth dataset:** Manual — 10 queries covering specific subroutine lookups, conceptual queries, cross-cutting queries, and dependency queries.
- **Realistic for timeline:** 10 queries is achievable in 24 hours. Expand to 20+ post-MVP.
- **User feedback:** Not for MVP. Thumbs up/down on answers is a stretch goal.

14. Performance Optimization

- **Ingestion resilience:** Cache embedding responses to a local .jsonl file during ingestion. This avoids re-spending tokens on identical text if the script restarts.
- **Index optimization:** Pinecone handles index optimization automatically. Cosine similarity metric. No custom configuration needed at this scale.
- **Query preprocessing:** Lowercase and trim whitespace. No stemming or lemmatization — the embedding model handles semantic similarity natively.
- **Query caching (post-MVP):** Implement server-side caching (Redis or KV store) for repeated queries once the web UI is added.

15. Observability

- **Logging:** Standard Python logging for parsed queries, top-k results, token usage, and total latency.
- **Metrics to track:** Query latency, average top-k similarity scores, API error rates, queries per session.
- **Alerting:** None for a sprint project. Monitor logs manually.

16. Deployment & DevOps

- **MVP deployment:** Local CLI. Run `python query.py "your question"`.
 - **Post-MVP deployment:** Next.js on Vercel with serverless functions.
 - **Secrets handling:** API keys via `.env` file and `python-dotenv`.
 - **Environment management:** `requirements.txt` for Python dependencies.
 - **CI/CD for index updates:** None needed for static LAPACK. Re-run ingestion manually if needed.
-

AI Cost Analysis

Production Cost Projections (Post-MVP Reference)

Assumptions: 5 queries/user/day, 30 days/month, ~3K input + ~500 output tokens per LLM call.

Component	100 Users	1,000 Users	10,000 Users	100,000 Users
Queries/month	15,000	150,000	1,500,000	15,000,000
LLM (GPT-4o-mini)	\$11	\$113	\$1,125	\$11,250
Embedding (queries)	<\$1	<\$1	\$1	\$6
Pinecone	\$0 (free)	\$70 (Standard)	\$200	\$500+
Hosting	\$0 (local)	\$20 (Vercel Pro)	\$20	\$150+
Total/month	~\$12	~\$204	~\$1,346	~\$11,906

Key insight: LLM calls dominate at scale (95% of spend at 100K users).

Architecture Summary

```
graph TD
  subgraph "Python Ingestion (local, one-time)"
    A[Scan .f files] --> B[Parse SUBROUTINE]
    B --> C[Extract metadata]
    C --> D[Embed chunks]
    D --> E[Cache to .jsonl]
    E --> F[Upsert to Pinecone]
  end

  subgraph "CLI / Web App (local / Vercel)"
    G[User query] --> H[Embed query]
    H -- OpenAI embeddings --> H
    H --> I[Search]
    I -- Pinecone --> I
    I --> J[Top-k chunks score > 0.7]
    J --> K[LLM answer]
    K -- OpenAI GPT-4o-mini --> K
    K --> L[Stream response + cited code snippets]
  end

  F -. -> I
```